

In this lesson, you can find the full source code for the project used within the course. Feel free to use this lesson as needed – whether to help you debug your code, use it as a reference later, or expand the project to practice your skills!

Dev Shop

AGENTS.md

Found in folder: /

This file documents the project's setup, technical stack, development commands, and coding conventions. It establishes strict rules for state management, dependencies, and testing to maintain the project as a controlled learning environment. The rules are designed to keep the codebase simple and focused for students practicing AI-assisted development.

AGENTS.md

Project

Dev Shop — a React shopping cart starter project **for** a coding course on AI-assisted development. Students use **this** as a base to practice adding features with AI assistance.

The app shows a product list on the left and a cart sidebar on the right. All data is hardcoded; there is no backend, API, or localStorage.

Stack

- React 19 (functional components only)
- Vite 7
- CSS Modules (no Tailwind, no external UI libraries)
- Vitest + @testing-library/react **for** tests

Commands

Command	Purpose
---------	---------

---	---
-----	-----

`npm run dev`	Start dev server (hot reload)
---------------	-------------------------------

`npm run build`	Production build
-----------------	------------------

`npm run preview`	Preview production build
-------------------	--------------------------

`npm run test`	Run test suite (Vitest, non-watch)
----------------	------------------------------------

Conventions

- **Functional components only** — no **class** components
- **CSS Modules only** — one `.module.css` per component, **class** names in camelCase
- **No external UI libraries** — style from scratch
- **Props down, events up** — all cart state lives in `App.jsx` and is passed to children as props; children emit events via callback props (`onAddToCart`, `onRemove`, `onClearCart`)
- **No routing** — single-page, no React Router
- **Derived values in render** — line totals and cart total are computed inline, not stored in state
- **Immutable state updates** — always **return new** arrays from `setCartItems`, never mutate

Critical Rules

1. **Cart state must stay in `App.jsx`**. **Do** not lift state into a context, store, or separate hook unless explicitly asked. **This** is a teaching project — the simple prop-drilling pattern is intentional.
2. **Do not add `localStorage`, `sessionStorage`, or any persistence.** State resets on page reload by design.
3. **Do not install additional dependencies** beyond Vite, React, Vitest, and `@testing-library/react`. Students should see a minimal, legible dependency tree.
4. **Missing features are intentional.** **This** is a starter project. **If** something seems unimplemented (e.g., quantity controls, checkout flow), assume it is a student exercise — **do** not silently add it.
5. **All 10 tests must pass** after any change. Run `npm run test` to verify before considering work complete.
6. **ALWAYS run `npm run test` after making any changes to component logic or state management.** **Do** not consider a task complete until all tests pass.

README.md

Found in folder: /

This file is a README that documents a React shopping cart project. It describes the app's features, tech stack, setup instructions, and project structure. All data is hardcoded with no external dependencies.

Dev Shop — React Shopping Cart

A simple shopping cart app built with React and Vite. Designed as a starter project **for** a coding course on AI-assisted development.

What it does

- Displays a product catalog on the left
- Lets users add products to a cart sidebar on the right
- Tracks quantities when the same product is added more than once
- Shows per-item line totals and a cart grand total
- Supports removing individual items or clearing the cart entirely

All data is hardcoded — no backend, no API calls, no localStorage.

Stack

- **React 19** — functional components, hooks
- **Vite** — dev server and build tool
- **CSS Modules** — scoped per-component styles, no Tailwind or UI libraries
- **Vitest** + **@testing-library/react** — unit/integration tests

Getting started

```
```bash
```

```
npm install
```

```
npm run dev
```

```
```
```

Running tests

```
```bash
```

```
npm run test
```

`, `

The test suite has 10 tests covering product rendering, add-to-cart, quantity tracking, line totals, remove, clear cart, and empty-cart state.

## ## Project structure

`, `

src/

components/

ProductList.jsx + ProductList.module.css

ProductCard.jsx + ProductCard.module.css

Cart.jsx + Cart.module.css

CartItem.jsx + CartItem.module.css

data/

products.js # hardcoded product catalog

App.jsx # root component, owns all cart state

App.module.css

main.jsx

setupTests.js # @testing-library/jest-dom setup

docs/

ui.md # styling conventions

state.md # state structure and rules

`, `

## ## Products

| Name | Price |

|---|---|

| Mechanical Keyboard | \$89.99 |

| USB-C Hub | \$34.99 |

| Monitor Stand | \$49.99 |

| Webcam HD 1080p | \$79.99 |

| Desk Lamp LED | \$29.99 |

...

## state.md

Found in folder: **/docs**

This document outlines the state management architecture for a shopping cart feature. It specifies that all cart state is centralized in a parent App component, which passes data and callbacks down to child components. The rules define how state is updated immutably and how derived values like totals are calculated.

# State Management

## Where state lives

**\*\*All cart state lives in `App.jsx`.** No state **in** child components. No external store.

...

App (owns cartItems state)

└─ ProductList ← receives products + onAddToCart prop

| └─ ProductCard ← receives product + onAddToCart prop

└─ Cart ← receives cartItems + onRemove + onClearCart props

└─ CartItem ← receives item + onRemove props

...

## State shape

```js

cartItems: [

{ id: number, name: string, price: number, quantity: number },

...

]

```

Each item **in** `cartItems`` is a product object extended with a `quantity`` field. Products themselves are never mutated — cart items are copies.

## ## State transitions

Action	Handler	Result
--------	---------	--------

---	---	---
-----	-----	-----

Add <b>new</b> product	<code>handleAddToCart(product)`</code>	Appends <code>{ ...product, quantity: 1 }`</code>
------------------------	----------------------------------------	---------------------------------------------------

Add existing product	<code>handleAddToCart(product)`</code>	Increments <code>quantity`</code> on matching item
----------------------	----------------------------------------	----------------------------------------------------

Remove item	<code>handleRemove(productId)`</code>	Filters out item with matching <code>id`</code>
-------------	---------------------------------------	-------------------------------------------------

Clear cart	<code>handleClearCart()`</code>	Resets <code>cartItems`</code> to <code>[]`</code>
------------	---------------------------------	----------------------------------------------------

## ## Rules

- **\*\*Never mutate state.\*\*** Always **return** a **new** array from `setCartItems``.

- **\*\*No localStorage\*\***, no sessionStorage, no backend sync — state is **in**-memory only and resets on page reload.

- **\*\*Derived values\*\*** (line totals, cart total) are computed inline **in** the render — not stored **in** state.

- **\*\*Props down, events up\*\*** — child components receive data and callbacks; they never reach up to modify parent state directly.

## ## Computed values

Value	Computed in	Formula
-------	-------------	---------

---	---	---
-----	-----	-----

Line total	<code>CartItem.jsx`</code>	<code>item.price * item.quantity`</code>
------------	----------------------------	------------------------------------------

Cart total	<code>Cart.jsx`</code>	<code>cartItems.reduce((sum, item) =&gt; sum + item.price * item.quantity, 0)`</code>
------------	------------------------	---------------------------------------------------------------------------------------

## ui.md

Found in folder: **/docs**

This markdown file defines the styling conventions and design system for the project. It specifies that CSS Modules are used exclusively for component-scoped styling, with no

utility frameworks or external UI libraries. The document outlines layout structure, color palette, typography rules, component styling guidelines, and CSS naming conventions.

## # UI & Styling Conventions

### ## Approach

All styling uses **CSS Modules** (`*.module.css`). No Tailwind, no utility classes, no external UI libraries.

Each component owns its styles **in** a sibling `.module.css` file.

### ## Layout

The app uses a two-column layout defined **in** `App.module.css`:

- Left column — `ProductList`` (fluid, `flex: 1``)
- Right column — `Cart`` sidebar (fixed width `320px``)

The outer wrapper is a flex row with `gap: 24px`` and `padding: 32px``.

### ## Color palette

Role	Value
------	-------

---	---
-----	-----

Page background	<code>#edf2f7`</code>
-----------------	-----------------------

Card / sidebar background	<code>fff` / #f7fafc`</code>
---------------------------	------------------------------

Border	<code>#e2e8f0`</code>
--------	-----------------------

Primary text	<code>#1a202c`</code>
--------------	-----------------------

Secondary text	<code>#4a5568`</code>
----------------	-----------------------

Muted text	<code>#718096`</code>
------------	-----------------------

Primary action (button)	<code>#3182ce`</code> hover <code>#2b6cb0`</code>
-------------------------	---------------------------------------------------

Destructive (remove)	<code>#e53e3e`</code>
----------------------	-----------------------

### ## Typography

- Base font: system font stack (inherited from browser default)
- Headings use `font-weight: 700``
- Button labels use `font-weight: 500``

- No custom font imports

## ## Component style rules

- Cards and sidebars use `border-radius: 8px` and `border: 1px solid #e2e8f0`
- Buttons use `border-radius: 6px` (primary) or `border-radius: 4px` (small/secondary)
- Transitions on interactive elements: `0.15s` duration on `background` and `color`
- No box-shadows — borders only

## ## Naming convention

CSS Module **class** names use camelCase (e.g., `.card`, `.addButton`, `.lineTotal`). Keep names descriptive and scoped to the component — no global **class** names.

## App.jsx

Found in folder: **/src**

This code defines the main App component for an e-commerce interface. It manages a shopping cart's state, allowing users to add items, remove individual items, and clear the entire cart. It renders a product list and a cart sidebar, passing down the necessary state and handler functions.

```
import { useState } from 'react';

import ProductList from './components/ProductList';

import Cart from './components/Cart';

import products from './data/products';

import styles from './App.module.css';

function App() {

 // State to hold the array of items currently in the shopping cart

 const [cartItems, setCartItems] = useState([]);

 // Adds a product to the cart or increments its quantity if already present

 function handleAddToCart(product) {

 setCartItems((prev) => {

 const existing = prev.find((item) => item.id === product.id);
```

```

 if (existing) {

 // Increment quantity for the existing item

 return prev.map((item) =>

 item.id === product.id ? { ...item, quantity: item.quantity + 1 } : item

);

 }

 // Add new item to cart with initial quantity of 1

 return [...prev, { ...product, quantity: 1 }];

});

}

// Removes an item from the cart based on its product ID

function handleRemove(productId) {

 setCartItems((prev) => prev.filter((item) => item.id !== productId));

}

// Clears all items from the cart

function handleClearCart() {

 setCartItems([]);

}

return (

 <div className={styles.layout}>

 <header className={styles.header}>

 <h1 className={styles.title}>Dev Shop</h1>

 </header>

 <main className={styles.main}>

 {/* ProductList displays all available products */}

 <ProductList products={products} onAddToCart={handleAddToCart} />

```

```

 {/* Cart displays current cart items and provides removal/clear actions */}

 <Cart
 cartItems={cartItems}
 onRemove={handleRemove}
 onClearCart={handleClearCart}
 />

 </main>

</div>

);
}

export default App;

```

### **App.test.jsx**

Found in folder: **/src**

This code tests a React shopping cart application. It verifies that product listings display correctly and that cart interactions like adding items, calculating totals, applying discounts, and clearing the cart work as expected. The tests simulate user actions to check the UI updates appropriately.

```

import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { describe, it, expect } from 'vitest';
import App from './App';

// Helper function to initialize user events and render the App component

function setup() {
 const user = userEvent.setup();
 render(<App />);
 return { user };
}

```

```

describe('Product list', () => {
 it('renders all 5 products', () => {
 setup();

 // Find all "Add to cart" buttons to count products
 const buttons = screen.getAllByRole('button', { name: /add to cart/i });
 expect(buttons).toHaveLength(5);
 });

 it('shows each product name and price', () => {
 setup();

 // Assert that all expected product names and prices are displayed
 expect(screen.getByText('Mechanical Keyboard')).toBeInTheDocument();
 expect(screen.getByText('$89.99')).toBeInTheDocument();
 expect(screen.getByText('USB-C Hub')).toBeInTheDocument();
 expect(screen.getByText('$34.99')).toBeInTheDocument();
 expect(screen.getByText('Monitor Stand')).toBeInTheDocument();
 expect(screen.getByText('$49.99')).toBeInTheDocument();
 expect(screen.getByText('Webcam HD 1080p')).toBeInTheDocument();
 expect(screen.getByText('$79.99')).toBeInTheDocument();
 expect(screen.getByText('Desk Lamp LED')).toBeInTheDocument();
 expect(screen.getByText('$29.99')).toBeInTheDocument();
 });
});

describe('Add to cart', () => {
 it('adds a product to the cart when "Add to Cart" is clicked', async () => {
 const { user } = setup();

 // Click the first "Add to cart" button

```

```

 await user.click(screen.getAllByRole('button', { name: /add to cart/i })[0]);

 // Check that the product name appears in the cart (within a span element)
 expect(screen.getByText('Mechanical Keyboard', { selector: 'span'
 })).toBeInTheDocument();

 });

 it('increases quantity to 2 when the same product is added twice', async () => {

 const { user } = setup();

 const addButton = screen.getAllByRole('button', { name: /add to cart/i })[0];

 await user.click(addButton);

 await user.click(addButton);

 // Expect to see a quantity indicator like "× 2"
 expect(screen.getByText(/× 2/)).toBeInTheDocument();

 });

 it('shows correct line total for a product with quantity 2', async () => {

 const { user } = setup();

 const addButton = screen.getAllByRole('button', { name: /add to cart/i })[0];

 await user.click(addButton);

 await user.click(addButton);

 // Mechanical Keyboard $89.99 × 2 = $179.98 (line total and original total are both
 $179.98)

 expect(screen.getAllByText('$179.98')).toHaveLength(2);

 });

 it('shows correct total when multiple different products are added', async () => {

 const { user } = setup();

 const addButtons = screen.getAllByRole('button', { name: /add to cart/i });

 // Mechanical Keyboard $89.99 + USB-C Hub $34.99 = $124.98

 await user.click(addButtons[0]);

```

```

 await user.click(addButtons[1]);

 expect(screen.getByText(/\$124\.98/)).toBeInTheDocument();
 });

 it('does not show a discount when the cart total is $100 or less', async () => {
 const { user } = setup();

 await user.click(screen.getAllByRole('button', { name: /add to cart/i })[0]);

 expect(screen.getByText('Original total:')).toBeInTheDocument();

 expect(screen.getByText('Total:')).toBeInTheDocument();

 // Discount section should not appear for totals ≤ $100

 expect(screen.queryByText('Discount:')).not.toBeInTheDocument();
 });

 it('applies a 10% discount when the cart total exceeds $100', async () => {
 const { user } = setup();

 const addButtons = screen.getAllByRole('button', { name: /add to cart/i });

 await user.click(addButtons[0]);

 await user.click(addButtons[1]);

 expect(screen.getByText('Original total:')).toBeInTheDocument();

 expect(screen.getByText('$124.98')).toBeInTheDocument();

 expect(screen.getByText('Discount:')).toBeInTheDocument();

 expect(screen.getByText('- $12.50')).toBeInTheDocument();

 expect(screen.getByText('$112.48')).toBeInTheDocument();
 });
});

describe('Cart interactions', () => {
 it('removes an item entirely when "Remove" is clicked', async () => {
 const { user } = setup();

```

```

 await user.click(screen.getAllByRole('button', { name: /add to cart/i })[0]);

 // Click the "Remove" button for the added item
 await user.click(screen.getByRole('button', { name: /remove/i }));

 // Verify the product is no longer in the cart
 expect(screen.queryByText('Mechanical Keyboard', { selector: 'span'
 })).not.toBeInTheDocument();

 });

 it('empties the cart when "Clear Cart" is clicked', async () => {
 const { user } = setup();

 const addButtons = screen.getAllByRole('button', { name: /add to cart/i });

 await user.click(addButtons[0]);
 await user.click(addButtons[1]);

 await user.click(screen.getByRole('button', { name: /clear cart/i }));

 // Check that the empty cart message appears
 expect(screen.getByText(/your cart is empty/i)).toBeInTheDocument();

 });

});

describe('Empty cart state', () => {
 it('shows "Your cart is empty" when no items are in the cart', () => {
 setup();

 expect(screen.getByText(/your cart is empty/i)).toBeInTheDocument();

 });

 it('does not show "Your cart is empty" when the cart has items', async () => {
 const { user } = setup();

 await user.click(screen.getAllByRole('button', { name: /add to cart/i })[0]);

 // The empty message should disappear after adding an item

```

```
 expect(screen.queryByText(/your cart is empty/i)).not.toBeInTheDocument();
 });
});
```

### **main.jsx**

Found in folder: **/src**

This code initializes a React application by rendering the main App component inside a React StrictMode wrapper into the HTML element with the id 'root'. It uses React 18's createRoot API for concurrent rendering features. The StrictMode enables additional development checks for potential problems.

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App.jsx';

// Initialize React root on the DOM element with id 'root'
createRoot(document.getElementById('root')).render(
 <StrictMode>
 { /* Main application component wrapped in StrictMode for development checks */ }
 <App />
 </StrictMode>
);
```

### **Cart.jsx**

Found in folder: **/src/components**

This component renders a shopping cart sidebar. It displays a list of cart items, calculates a total and a 10% discount if the total exceeds \$100, and shows a summary. It also provides a button to clear all items from the cart.

```
import CartItem from './CartItem';
import styles from './Cart.module.css';

function Cart({ cartItems, onRemove, onClearCart }) {
```

```

// Calculate the total price of all items in the cart
const total = cartItems.reduce((sum, item) => sum + item.price * item.quantity, 0);

// Apply a 10% discount if the total exceeds $100
const discountAmount = total > 100 ? total * 0.1 : 0;
const discountedTotal = total - discountAmount;

return (
 <aside className={styles.sidebar}>
 <h2 className={styles.heading}>Cart</h2>
 {/* Conditionally render empty message or cart contents */}
 {cartItems.length === 0 ? (
 <p className={styles.empty}>Your cart is empty</p>
) : (
 <>
 <div className={styles.items}>
 {/* Render each cart item as a separate CartItem component */}
 {cartItems.map((item) => (
 <CartItem key={item.id} item={item} onRemove={onRemove} />
))}
 </div>
 <div className={styles.footer}>
 <div className={styles.summary}>
 <p className={styles.summaryRow}>
 Original total: ${total.toFixed(2)}
 </p>
 {/* Only show discount line if a discount was applied */}
 {discountAmount > 0 && (

```

```

 <p className={styles.summaryRow}>
 Discount: -${discountAmount.toFixed(2)}
 </p>
)}
 <p className={styles.total}>
 Total: ${discountedTotal.toFixed(2)}
 </p>
</div>

<button className={styles.clear} onClick={onClearCart}>
 Clear Cart
</button>
</div>
</>
)}
</aside>
);
}

export default Cart;

```

### **CartItem.jsx**

Found in folder: **/src/components**

This component renders a single item in a shopping cart, displaying its name, price, quantity, and calculated total. It provides a button to remove the item from the cart by calling the provided `onRemove` function with the item's ID.

```
import styles from './CartItem.module.css';
```

```
function CartItem({ item, onRemove }) {
```

```
 // Calculate the total cost for this line item
```

```
 const lineTotal = item.price * item.quantity;
```

```

return (
 <div className={styles.item}>
 <div className={styles.details}>
 {item.name}

 ${item.price.toFixed(2)} × {item.quantity} ={' '}
 ${lineTotal.toFixed(2)}

 </div>
 { /* Calls onRemove with the item's id when clicked */ }
 <button className={styles.remove} onClick={() => onRemove(item.id)}>
 Remove
 </button>
 </div>
);
}

```

export default CartItem;

### ProductCard.jsx

Found in folder: **/src/components**

This code defines a React component that displays a product's name and price in a styled card, and includes a button that triggers a function to add that product to a cart when clicked.

```
import styles from './ProductCard.module.css';
```

```
// Component to display a single product's information and an action button.
```

```
function ProductCard({ product, onAddToCart }) {
```

```
 return (
 <div className={styles.card}>
```

```

<div className={styles.info}>

 <h4 className={styles.name}>{product.name}</h4>

 <p className={styles.price}>${product.price.toFixed(2)}</p> {/* Formats price to two
decimal places */}

</div>

<button className={styles.button} onClick={() => onAddToCart(product)}>

 Add to Cart

</button>

</div>

);
}

```

export default ProductCard;

### ProductList.jsx

Found in folder: **/src/components**

This code renders a list of products by mapping over a provided array. It displays each product using a `ProductCard` component, passing along product data and an add-to-cart handler. The layout is styled using CSS modules.

```

import ProductCard from './ProductCard';

import styles from './ProductList.module.css';

// Renders a grid of product cards

function ProductList({ products, onAddToCart }) {

 return (

 <section className={styles.list}>

 <h2 className={styles.heading}>Products</h2>

 <div className={styles.grid}>

 {/* Iterate over products array to create a card for each */}

 {products.map((product) => (

```

```

 <ProductCard
 key={product.id} // Unique key for React list rendering
 product={product}
 onAddToCart={onAddToCart}
 />
)))
</div>

</section>

);
}

export default ProductList;

```

## index.html

Found in folder: /

This HTML file sets up the basic structure of a web page, including metadata and viewport settings for responsive design. It links to a favicon and loads a React application by mounting it into a `div` element with the id “root”. The page’s JavaScript entry point is a module script located at ` /src/main.jsx` .

```

<!doctype html>

<html lang="fr">

 <head>

 <meta charset="UTF-8" />

 <link rel="icon" type="image/svg+xml" href="/vite.svg" />

 <meta name="viewport" content="width=device-width, initial-scale=1.0" />

 <title>Vite + React</title>

 </head>

 <body>

 <!-- Root container where the React app will be rendered -->

```

```
<div id="root"></div>

<!-- Entry point for the application's JavaScript, using ES modules -->

<script type="module" src="/src/main.jsx"></script>

</body>

</html>
```

### **App.module.css**

Found in folder: **/src**

This CSS defines a responsive page layout with a dark header and a light flex container. It uses a column flexbox for the main content area and adjusts padding and orientation on smaller screens. The media query changes the layout to a vertical stack on mobile devices.

```
.layout {
 min-height: 100vh; /* Full viewport height minimum */
 background: #edf2f7; /* Light gray background */
 display: flex;
 flex-direction: column; /* Stack children vertically */
}

.header {
 background: #1a202c; /* Dark header background */
 padding: 16px 32px;
}

.title {
 margin: 0;
 font-size: 1.5rem;
 font-weight: 700;
 color: #fff; /* White text */
 letter-spacing: 0.025em;
}
```

```
.main {
 display: flex;
 align-items: flex-start; /* Align items to the top */
 gap: 24px; /* Space between flex children */
 padding: 32px;
 flex: 1; /* Take remaining vertical space */
}

@media (max-width: 640px) {
 .header {
 padding: 12px 16px; /* Reduced padding on mobile */
 }
 .main {
 flex-direction: column; /* Stack items vertically on small screens */
 padding: 16px;
 gap: 16px;
 }
}
```

### **Cart.module.css**

Found in folder: **/src/components**

This CSS styles a sidebar component with a clean, responsive design. It defines visual styles for elements like headings, items, summaries, and a clear button, including hover effects. On smaller screens, the sidebar adjusts to full width and changes its alignment.

```
.sidebar {
 width: 320px;
 flex-shrink: 0; /* Prevents sidebar from shrinking in a flex container */
 background: #f7fafc;
 border: 1px solid #e2e8f0;
```

```
border-radius: 8px;

padding: 20px;

display: flex;

flex-direction: column;

align-self: flex-start; /* Aligns to the top of a flex container */
}

@media (max-width: 640px) {

 .sidebar {

 width: 100%; /* Makes sidebar responsive on small screens */

 align-self: stretch; /* Fills available width */

 }

}

.heading {

 margin: 0 0 16px;

 font-size: 1.5rem;

 font-weight: 700;

 color: #1a202c;

}

.empty {

 color: #718096;

 font-size: 0.9rem;

 margin: 0;

}

.items {

 display: flex;

 flex-direction: column;
```

```
}

.footer {
 margin-top: 16px;
 display: flex;
 flex-direction: column;
 gap: 12px; /* Spacing between child elements */
}

.summary {
 display: flex;
 flex-direction: column;
 gap: 6px;
}

.summaryRow,
.total {
 margin: 0;
 font-size: 1rem;
 color: #1a202c;
}

.summaryRow {
 display: flex;
 justify-content: space-between;
 gap: 12px;
}

.summaryRow:nth-child(2) {
 color: #2f855a; /* Highlights a specific summary row, e.g., for discounts */
}
```

```
.total{
 display: flex;
 justify-content: space-between;
 gap: 12px;
 padding-top: 8px;
 border-top: 1px solid #e2e8f0; /* Visual separator for total row */
}

.clear{
 padding: 8px 16px;
 background: transparent;
 color: #718096;
 border: 1px solid #cbd5e0;
 border-radius: 6px;
 font-size: 0.875rem;
 cursor: pointer;
 transition: background 0.15s, border-color 0.15s; /* Smooth hover transition */
}

.clear:hover{
 background: #edf2f7;
 border-color: #a0aec0;
}
```

### **CartItem.module.css**

Found in folder: **/src/components**

This CSS styles a list item component with a flexible layout, including details and a remove button. It defines visual styles for text elements and an interactive button with hover effects. The last item's bottom border is removed for cleaner spacing.

```
.item{
```

```
display: flex;

align-items: flex-start;

justify-content: space-between;

padding: 12px 0;

border-bottom: 1px solid #e2e8f0;

gap: 8px;
}

.item:last-of-type {

border-bottom: none; /* Removes border from the last item */
}

.details {

display: flex;

flex-direction: column;

gap: 4px;

flex: 1; /* Allows details to take available space */

min-width: 0; /* Prevents overflow in flex container */
}

.name {

font-size: 0.9rem;

font-weight: 600;

color: #1a202c;
}

.meta {

font-size: 0.8rem;

color: #4a5568;
}
```

```
.remove {
 padding: 4px 10px;
 background: transparent;
 color: #e53e3e;
 border: 1px solid #e53e3e;
 border-radius: 4px;
 font-size: 0.75rem;
 cursor: pointer;
 white-space: nowrap; /* Prevents button text from wrapping */
 transition: background 0.15s, color 0.15s;
}

.remove:hover {
 background: #e53e3e;
 color: #fff;
}
```

### **ProductCard.module.css**

Found in folder: **/src/components**

This CSS defines styles for a product card component, including layout, typography, and interactive button states. It creates a flex container for the card with spaced alignment and a hover effect for the button. The styles set visual properties like colors, borders, and spacing for the card's elements.

```
.card {
 display: flex;
 align-items: center;
 justify-content: space-between;
 padding: 16px;
 background: #fff;
```

```
border: 1px solid #e2e8f0;

border-radius: 8px;

gap: 12px; /* Space between child elements */
}

.info {

display: flex;

flex-direction: column;

gap: 4px; /* Vertical spacing between name and price */
}

.name {

margin: 0;

font-size: 1rem;

font-weight: 600;

color: #1a202c;
}

.price {

margin: 0;

font-size: 0.9rem;

color: #4a5568;
}

.button {

padding: 8px 16px;

background: #3182ce;

color: #fff;

border: none;

border-radius: 6px;
```

```
font-size: 0.875rem;

font-weight: 500;

cursor: pointer;

white-space: nowrap; /* Prevents button text from wrapping */

transition: background 0.15s; /* Smooth color change on hover */
}

.button:hover {

 background: #2b6cb0; /* Darker blue on hover */
}
```

### **ProductList.module.css**

Found in folder: **/src/components**

This CSS defines three classes: `.list`` sets flexible sizing with a minimum width, `.heading`` styles a bold, large, dark header with bottom margin, and `.grid`` creates a vertical flex container with spacing between items.

```
.list {

 flex: 1; /* Allows element to grow/shrink, taking available space */

 min-width: 0; /* Prevents flex item from overflowing its container */
}

.heading {

 margin: 0 0 16px; /* Top:0, Right:0, Bottom:16px, Left:0 */

 font-size: 1.5rem;

 font-weight: 700;

 color: #1a202c; /* Dark gray color */
}

.grid {

 display: flex;

 flex-direction: column; /* Stack children vertically */
}
```

```
gap: 12px; /* Space between child elements */
}
```

## **products.js**

Found in folder: **/src/data**

This code defines an array of product objects with properties like id, name, and price, then exports it as the default export for use in other modules.

```
const products = [
 { id: 1, name: 'Mechanical Keyboard', price: 89.99 },
 { id: 2, name: 'USB-C Hub', price: 34.99 },
 { id: 3, name: 'Monitor Stand', price: 49.99 },
 { id: 4, name: 'Webcam HD 1080p', price: 79.99 },
 { id: 5, name: 'Desk Lamp LED', price: 29.99 },
];

// Export the products array as the default module export
export default products;
```

## **Finance Tracker**

### **AGENTS.md**

Found in folder: /

This document outlines the project structure, technical stack, and development rules for a personal finance tracker. It specifies the commands, file naming conventions, and strict data flow where all business logic is isolated to a service layer. The rules enforce a static, hardcoded implementation without external dependencies or state persistence.

# Finance Tracker — AGENTS.md

## Project

A personal finance tracker built as a starter project **for** a coding course on AI-assisted development. Displays a list **of** transactions, a summary dashboard with totals by category, and a monthly balance view. All data is hardcoded — no backend, no API, no localStorage.

## Stack

| Layer | Technology |

|---|---|

| Framework | React 19 + Vite 7 |

| Styling | CSS Modules (plain CSS, no Tailwind) |

| Testing | Vitest 4 + @testing-library/react |

| Linting | ESLint 9 (flat config) |

| Language | JavaScript (ESM) |

## Commands

```bash

npm run dev # start dev server at http://localhost:5173

npm run build # production build → dist/

npm run preview # preview the production build

npm run lint # lint src/ with ESLint

npm run test # run Vitest in watch mode

npm run test:run # run tests once (CI mode)

```

## Conventions

### File structure

```

src/

components/ # React components, one per file + matching .module.css

services/ # All business logic and data access

utils/ # Pure utility functions (formatting, colour mapping)

data/ # Hardcoded mock data

App.jsx # Root component — wires service layer to components

main.jsx # React entry point

...

Naming

- Components: PascalCase (`TransactionItem.jsx`)
- CSS modules: same name as component (`TransactionItem.module.css`)
- Services and utils: camelCase (`transactionService.js` , `currencyUtils.js`)
- Test files: co-located with the module they test, ` .test.js ` suffix

CSS

- Plain CSS modules only — no Tailwind, no external UI libraries
- Colour palette: slate grays **for** text (` #0f172a` , ` #1e293b` , ` #64748b`), green ` #16a34a ` for income, red ` #dc2626 ` for expenses
- Component styles are scoped; global resets live **in** ` App.module.css ` only

Data flow

- Service functions **in** ` transactionService.js ` are the single source **of** truth
- ` App.jsx ` calls the service layer once at module load (not inside render)
- Components receive data via props — they never import from services or data files directly

Critical Rules

1. ****All calculations must happen in ` transactionService.js `**** — components only receive and display data. Never put filtering, summing, or aggregation logic inside a component.
2. ****No new dependencies**** — the allowed set is: React, ReactDOM, Vite, Vitest, @testing-library/react, and ESLint. **Do** not add UI libraries, date libraries, or state managers.
3. ****No localStorage, no fetch, no backend**** — **this** is a fully static, hardcoded project.
4. ****12 tests must stay green**** — **do** not modify the test files without keeping all 12 passing.
5. ****` npm run lint ` must pass with zero errors**** — fix lint errors before committing.
6. ****The project contains intentional bugs**** seeded **for** student exercises — **do** not silently fix code that looks wrong without reading the course context first.

README.md

Found in folder: /

This Markdown file serves as the project documentation for a personal finance tracker application. It outlines the application's features, setup instructions, project structure, and technical architecture. The document is intended to guide developers on how to run, build, and understand the codebase.

Finance Tracker

A React + Vite personal finance tracker built as a starter project **for** a coding course on AI-assisted development.

What it does

- Displays 12 hardcoded transactions **for** January 2025
- Shows a summary dashboard with total income, total expenses, and current balance
- Breaks down spending by category with colour-coded labels
- Highlights whether the month is **in** profit or loss

Getting started

```
```bash
```

```
npm install
```

```
npm run dev
```

```
```
```

Open [<http://localhost:5173>](http://localhost:5173).

Scripts

| Command | Description |
|---------|-------------|
|---------|-------------|

| | |
|-----|-----|
| --- | --- |
|-----|-----|

| | |
|----------------------------|------------------------------|
| <code>`npm run dev`</code> | Start the development server |
|----------------------------|------------------------------|

| | |
|------------------------------|-----------------------------|
| <code>`npm run build`</code> | Build for production |
|------------------------------|-----------------------------|

| | |
|--------------------------------|------------------------------|
| <code>`npm run preview`</code> | Preview the production build |
|--------------------------------|------------------------------|

| | |
|-----------------------------|-------------------------------------|
| <code>`npm run lint`</code> | Lint <code>src/`</code> with ESLint |
|-----------------------------|-------------------------------------|

| `npm run test` | Run the Vitest test suite |

Project structure

...

src/

components/ React UI components (CSS Modules)

services/ Business logic — all calculations live here

utils/ Pure utility functions (currency formatting, colours)

data/ Hardcoded transaction data

App.jsx Root component

main.jsx Entry point

docs/

ui.md Styling conventions

state.md Data flow and services layer rules

testing.md Testing conventions

AGENTS.md Guide **for** Codex AI assistant

...

Tech stack

- **React 19** — UI

- **Vite 7** — build tool and dev server

- **CSS Modules** — scoped component styles, no external UI library

- **Vitest** — unit tests (12 tests covering services and utils)

- **ESLint 9** — flat config with React Hooks plugin

Design decisions

All data flows top-down: `data/transactions.js` → `services/transactionService.js` →

`App.jsx` → components via props. Components contain no business logic. **This** separation makes the service layer easy to test **in** isolation and straightforward to swap **for** a real API later.

state.md

Found in folder: **/docs**

This document outlines the data flow architecture and rules for a static financial application. It describes a unidirectional data flow from a hardcoded source, through a service layer for calculations, down to components via props. It establishes strict rules to keep business logic in services and display logic in components.

Data Flow & State

Architecture overview

...

src/data/transactions.js ← single source **of** truth (hardcoded array)

↓

src/services/transactionService.js ← all calculations and queries

↓

src/App.jsx ← calls service functions once at module load

↓

components (via props) ← display only, no logic

...

There is no state management library. There is no `useState` or `useReducer` **in this** project — data does not change at runtime, so all values are computed once and passed as props.

Services layer rules

All business logic lives **in** `src/services/transactionService.js`. The exported functions are:

| Function | Returns |

|---|---|

| `getAllTransactions()` | Full array **of** 12 transactions |

| `getTransactionsByCategory(category)` | Filtered array |

| `getTotalIncome()` | Number (positive) |

| `getTotalExpenses()` | Number (negative) |

| `getBalance()` | Number (income + expenses) |

| `getSummaryByCategory()` | `Array<{ category: string, total: number }>` |

****Rules:****

- Components must never import from `src/data/transactions.js` directly.
- Components must never perform filtering, summing, or aggregation.
- **If** you need a **new** derived value, add a **function** to `transactionService.js` and call it from `App.jsx`.

Props contract

`App.jsx` calls the service layer once and passes results down:

...

App

└─ TransactionList ({ transactions })

└─ TransactionItem ({ transaction }) ← per item

└─ Dashboard ({ income, expenses, balance, summary })

└─ MonthlyBalance ({ balance, month })

└─ CategorySummary ({ summary })

...

No component reaches up **for** data — data flows strictly downward via props.

Utils

Utils are pure functions with no side effects:

- `src/utils/currencyUtils.js` — `formatCurrency(amount)` → formatted string

- `src/utils/categoryUtils.js` — `getCategoryColour(category)` → hex colour string

Components may import utils directly since they are stateless helpers, not data sources.

File extension: md

testing.md

Found in folder: **/docs**

This markdown file documents the testing conventions for the project, including the framework used, file structure, import style, and a specific list of required tests with their expected outcomes.

Testing Conventions

Framework

[Vitest](<https://vitest.dev/>) — configured in `vitest.config.js` with the jsdom environment.

Run tests:

```
```bash
```

```
npm run test # watch mode
```

```
vitest --run # single pass (CI)
```

```
```
```

Test file location

Test files live alongside the module they test, using the `.test.js` suffix:

```
```
```

```
src/services/transactionService.js
```

```
src/services/transactionService.test.js
```

```
src/utils/currencyUtils.js
```

```
src/utils/currencyUtils.test.js
```

```
src/utils/categoryUtils.js
```

```
src/utils/categoryUtils.test.js
```

```
```
```

Import style

Always import Vitest globals explicitly — `globals: **false**` is set in `vitest.config.js`:

```
```js
```

```
import { describe, it, expect } from 'vitest';
```

```
```
```

What must be tested

The 12 required tests and what they verify:

| # | Test | Expected value |
|---|------|----------------|
|---|------|----------------|

| # | Test | Expected value |
|---|------|----------------|
|---|------|----------------|

| | | |
|---|-----------------------------------|--------------------------|
| 1 | <code>`getAllTransactions`</code> | <code>length 12</code> |
|---|-----------------------------------|--------------------------|

| | | |
|---|-------------------------------|-----------------------|
| 2 | <code>`getTotalIncome`</code> | <code> `4000`</code> |
|---|-------------------------------|-----------------------|

| | | |
|---|---------------------------------|------------------------|
| 3 | <code>`getTotalExpenses`</code> | <code> `-1493`</code> |
|---|---------------------------------|------------------------|

| | | |
|---|---------------------------|-----------------------|
| 4 | <code>`getBalance`</code> | <code> `2507`</code> |
|---|---------------------------|-----------------------|

| | | |
|---|--|----------------------------------|
| 5 | <code>`getTransactionsByCategory('Food')`</code> | <code> 3 items, all Food</code> |
|---|--|----------------------------------|

| | | |
|---|---|---------------------|
| 6 | <code>`getTransactionsByCategory('Unknown')`</code> | <code> `[]`</code> |
|---|---|---------------------|

| | | |
|---|-------------------------------------|-------------------------|
| 7 | <code>`getSummaryByCategory`</code> | <code>length 8</code> |
|---|-------------------------------------|-------------------------|

| | | |
|---|-------------------------------------|----------------------------------|
| 8 | <code>`getSummaryByCategory`</code> | <code>Food total `-230`</code> |
|---|-------------------------------------|----------------------------------|

| | | |
|---|-------------------------------------|----------------------------|
| 9 | <code>`formatCurrency(3200)`</code> | <code> `£3,200.00`</code> |
|---|-------------------------------------|----------------------------|

| | | |
|----|-------------------------------------|----------------------------|
| 10 | <code>`formatCurrency(-950)`</code> | <code> `-'£950.00`</code> |
|----|-------------------------------------|----------------------------|

| | | |
|----|----------------------------------|--|
| 11 | <code>`getCategoryColour`</code> | <code>for all 8 known categories returns a string</code> |
|----|----------------------------------|--|

| | | |
|----|---|---------------------------------------|
| 12 | <code>`getCategoryColour('Unknown')`</code> | <code> `'#6b7280'` (fallback)</code> |
|----|---|---------------------------------------|

Rules

- All 12 tests must pass before any PR is merged.

- **Do** not mock the data layer **in** service tests — tests should use the real ``transactions.js`` data so they **catch** regressions **in** both the data and the service functions.

- **Do** not test component rendering unless a **new** component test is specifically required — the current suite focuses on the service and util layers.

- **If** you add a **new** service **function**, add at least one test **for** it.

ui.md

Found in folder: **/docs**

This markdown file documents the UI and styling conventions for the project, including CSS module usage, layout structure, color palette, and typography rules. It establishes

consistent design patterns for components like cards and color-coded amounts. The guidelines ensure all styling is implemented with plain CSS according to the specified standards.

UI & Styling Conventions

CSS Modules

Every component has its own `.module.css` file with the same base name. Import styles as a default import named `styles`:

```
```.js

import styles from './ComponentName.module.css';

// use as: className={styles.container}

```
```

No Tailwind, no styled-components, no external UI library. All styling is plain CSS.

Layout

The root layout is a two-column CSS grid defined in `App.module.css`:

```
```.css

.layout {
 display: grid;
 grid-template-columns: 1fr 380px;
 gap: 24px;
 padding: 28px 32px;
}

```
```

- Left column: `TransactionList` (flexible width)
- Right column: `Dashboard` (fixed 380px)
- Collapses to single column below 768px

Colour palette

| Role | Value |

|---|---|

| Page background | `#f1f5f9` |

| Card background | `#ffffff` |

| Primary text | `#0f172a` |

| Secondary text | `#1e293b` |

| Muted text | `#64748b` |

| Subtle text / labels | `#94a3b8` |

| Border / divider | `#f1f5f9`, `#e2e8f0` |

| Income (green) | `#16a34a` |

| Expense (red) | `#dc2626` |

| Profit background | `#f0fdf4` |

| Loss background | `#fef2f2` |

Category colours are defined **in** `src/utils/categoryUtils.js` and returned by
`getCategoryColour(category)`. Never hardcode category colours **in** component files.

Card style

Panels (TransactionList, Dashboard) use:

```
` `` `css
```

```
background: #ffffff;
```

```
border-radius: 12px;
```

```
padding: 24px;
```

```
box-shadow: 0 1px 3px rgba(0, 0, 0, 0.08);
```

```
` `` `
```

Typography

- Font stack: ` -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif`

- Section headings: ` 1.1rem`, `font-weight: 700`

- Category/stat labels: ` 0.78–0.82rem`, uppercase, `letter-spacing: 0.05em`

- Transaction description: `0.95rem`, `font-weight: 500`

- Amount values: `0.95rem`, `font-weight: 600`

Colour-coding rule

- Any amount that is positive (income) → `#16a34a` green

- Any amount that is negative (expense) → `#dc2626` red

- Monthly balance card → green background/text **if** balance ≥ 0, red **if** < 0

File extension: md

MEMORY.md

Found in folder: **/memory**

This code provides a memory index linking to a project starter file for a personal finance tracker course. It notes the starter includes a full built project with 12 passing tests and instructs not to silently fix seeded bugs.

Memory Index

- **[Finance Tracker Course Starter](project_todo_course_starter.md)** — personal finance tracker starter; full project built, 12 tests passing; **do** not silently fix seeded bugs

App.jsx

Found in folder: **/src**

This code defines the main App component that renders a finance tracker interface. It imports transaction data and calculations from a service, then passes them to child components for display. The component structures the layout with a header and two main sections: a transaction list and a dashboard.

```
import styles from './App.module.css';
```

```
import TransactionList from './components/TransactionList.jsx';
```

```
import Dashboard from './components/Dashboard.jsx';
```

```
import {
```

```
  getAllTransactions,
```

```
  getTotalIncome,
```

```
  getTotalExpenses,
```

```

    getBalance,
    getExpenseSummaryByCategory,
  } from './services/transactionService.js';

  // Load transaction data and calculations once when module loads
  const transactions = getAllTransactions();
  const income = getTotalIncome();
  const expenses = getTotalExpenses();
  const balance = getBalance();
  const summary = getExpenseSummaryByCategory();

  function App() {
    return (
      <div className={styles.app}>
        <header className={styles.header}>
          <h1 className={styles.title}>Finance Tracker</h1>
          <p className={styles.subtitle}>January 2026</p>
        </header>
        <main className={styles.layout}>
          {/* Pass transaction data to list component */}
          <TransactionList transactions={transactions} />
          {/* Pass calculated financial data to dashboard component */}
          <Dashboard
            income={income}
            expenses={expenses}
            balance={balance}
            summary={summary}
          />

```

```
    </main>

  </div>

);

}

export default App;
```

main.jsx

Found in folder: **/src**

This code initializes a React application by rendering the main App component into the DOM element with the ID 'root'. It uses React's StrictMode to highlight potential problems during development. The App component is imported from a local file and wrapped in StrictMode for additional checks.

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App.jsx';

// Create a root for React to render into the DOM element with ID 'root'
const root = createRoot(document.getElementById('root'));

// Render the App component wrapped in StrictMode for development checks
root.render(

  <StrictMode>

    <App />

  </StrictMode>

);
```

CategorySummary.jsx

Found in folder: **/src/components**

This component displays a list of spending categories, their totals, and a visual warning for categories that have exceeded their budget. It uses a colored dot for each category and formats the currency values. An exclamation mark with a tooltip is shown next to categories that are over budget.

```

import styles from './CategorySummary.module.css';

import { formatCurrency } from '../utils/currencyUtils.js';
import { getCategoryColour } from '../utils/categoryUtils.js';

function CategorySummary({ summary }) {

  return (

    <div className={styles.container}>

      <h4 className={styles.heading}>By Category</h4>

      <ul className={styles.list}>

        {/* Render a list item for each category in the summary prop */}

        {summary.map((item) => (

          <li key={item.category} className={styles.row}>

            {/* Colored dot representing the category */}

            <span

              className={styles.dot}

              style={{ backgroundColor: getCategoryColour(item.category) }}

            />

            <span className={styles.label}>{item.category}</span>

            <span className={styles.total}>{formatCurrency(item.total)}</span>

            {/* Show warning icon if this category is over budget */}

            {item.isOverBudget && (

              <span

                className={styles.warning}

                aria-label={` ${item.category} is over budget by
${formatCurrency(item.overBudgetBy)} ` }

                title={` ${item.category} is over budget by ${formatCurrency(item.overBudgetBy)} ` }

              >

```

```

        !
      </span>
    })
  </li>
})
</ul>
</div>

);
}

export default CategorySummary;

```

Dashboard.jsx

Found in folder: **/src/components**

This code renders a dashboard sidebar component displaying financial overview data. It shows a monthly balance, total income and expenses in styled cards, and a category breakdown. The component uses imported helper functions and child components to format and present the data.

```

import styles from './Dashboard.module.css';

import CategorySummary from './CategorySummary.jsx';

import MonthlyBalance from './MonthlyBalance.jsx';

import { formatCurrency } from '../utils/currencyUtils.js';

function Dashboard({ income, expenses, balance, summary }) {

  return (

    <aside className={styles.container}>

      <h2 className={styles.heading}>Dashboard</h2>

      { /* Displays the balance for a specific month */ }

      <MonthlyBalance balance={balance} month="January" />

      <div className={styles.stats}>

```

```

    {/* Card for total income */}

    <div className={styles.statCard}>

      <p className={styles.statLabel}>Total Income</p>

      <p className={` ${styles.statValue}
${styles.income} `}>{formatCurrency(income)}</p>

    </div>

    {/* Card for total expenses */}

    <div className={styles.statCard}>

      <p className={styles.statLabel}>Total Expenses</p>

      <p className={` ${styles.statValue}
${styles.expense} `}>{formatCurrency(expenses)}</p>

    </div>

  </div>

  {/* Shows a breakdown of expenses or income by category */}

  <CategorySummary summary={summary} />

</aside>

);
}

```

export default Dashboard;

MonthlyBalance.jsx

Found in folder: **/src/components**

This component displays a monthly balance with conditional styling based on whether it's positive or negative. It shows the month name, formatted balance amount, and a profit/loss status label. The visual appearance changes between profit and loss states using different CSS classes.

```

import styles from './MonthlyBalance.module.css';

import { formatCurrency } from '../utils/currencyUtils.js';

```

```

function MonthlyBalance({ balance, month }) {

  const isProfit = balance >= 0; // Determine if balance is positive (profit) or negative (loss)

  return (

    // Apply different styling based on profit/loss status

    <div className={` ${styles.container} ${isProfit ? styles.profit : styles.loss}` >

      <p className={styles.label}>{month} Balance</p>

      <p className={styles.amount}>{formatCurrency(balance)}</p>

      <p className={styles.status}>{isProfit ? 'In profit' : 'In loss'}</p>

    </div>

  );
}

export default MonthlyBalance;

```

TransactionItem.jsx

Found in folder: **/src/components**

This component renders a single transaction as a list item, displaying its description, category, and formatted amount. It uses a colored dot to visually represent the transaction's category and styles the amount differently based on whether it is income or an expense.

```

import styles from './TransactionItem.module.css';

import { formatCurrency } from '../utils/currencyUtils.js';

import { getCategoryColour } from '../utils/categoryUtils.js';

function TransactionItem({ transaction }) {

  // Destructure transaction properties for easier access

  const { description, amount, category } = transaction;

  const isIncome = amount > 0; // Determine if transaction is income (positive amount)

  return (

    <li className={styles.item}>

```

```

    {/* Coloured dot representing the transaction category */}

    <span

      className={styles.dot}

      style={{ backgroundColor: getCategoryColour(category) }}

    />

    <div className={styles.details}>

      <span className={styles.description}>{description}</span>

      <span className={styles.category}>{category}</span>

    </div>

    {/* Amount styled differently for income vs expense */}

    <span className={` ${styles.amount} ${isIncome ? styles.income : styles.expense} `}>

      {formatCurrency(amount)}

    </span>

  </li>

);

}

```

export default TransactionItem;

TransactionList.jsx

Found in folder: **/src/components**

This component displays a list of transactions and provides a search input to filter them by description. It uses a state variable to track the search text and filters the transactions based on whether their description includes that text. The filtered list is then rendered using a separate `TransactionItem` component for each transaction.

```

import { useState } from 'react';

import styles from './TransactionList.module.css';

import TransactionItem from './TransactionItem.jsx';

function TransactionList({ transactions }) {

```

```
// State for the user's search input
const [searchText, setSearchText] = useState("");

// Normalize search text by trimming whitespace and converting to lowercase for
consistent comparison
const normalizedSearchText = searchText.trim().toLowerCase();

// Filter transactions based on search text; show all if search is empty
const visibleTransactions = normalizedSearchText

? transactions.filter((transaction) =>
    transaction.description.toLowerCase().includes(normalizedSearchText),
  )
: transactions;

return (
  <section className={styles.container}>
    <h2 className={styles.heading}>Transactions</h2>
    <label className={styles.searchLabel} htmlFor="transaction-search">
      Search transactions
    </label>
    <input
      className={styles.searchInput}
      id="transaction-search"
      type="search"
      value={searchText}
      onChange={(event) => setSearchText(event.target.value)}
      placeholder="Search by description"
    />
    <ul className={styles.list}>
```

```

    {visibleTransactions.map((transaction) => (
      <TransactionItem key={transaction.id} transaction={transaction} />
    )))
  </ul>
</section>

);
}

export default TransactionList;

```

index.html

Found in folder: /

This HTML file defines the basic structure of a web page, setting up metadata, linking resources, and creating a container where a React application will be mounted. It specifies the document type, language, character set, viewport for responsive design, and the page title. The `

` is the entry point for the React app, which is loaded by the script module `src/main.jsx`.

```

<!doctype html>

<html lang="fr">

  <head>

    <meta charset="UTF-8" />

    <link rel="icon" type="image/svg+xml" href="/vite.svg" />

    <meta name="viewport" content="width=device-width, initial-scale=1.0" />

    <title>Vite + React</title>

  </head>

  <body>

    <!-- Root element where the React application will be rendered -->

    <div id="root"></div>

    <!-- Entry point script for the React application built with Vite -->

```

```
<script type="module" src="/src/main.jsx"></script>

</body>

</html>
```

App.module.css

Found in folder: **/src**

This CSS file defines global and component-specific styles for a responsive web application layout. It sets a base style for all elements and styles for the header, title, and subtitle. It also creates a two-column grid layout for the main content that becomes a single column on smaller screens.

```
*{

  box-sizing: border-box; /* Ensures padding and border are included in element's total
width/height */

}

.app{

  min-height: 100vh; /* Makes the app container at least the full viewport height */

  background: #f1f5f9;

  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;

  color: #1e293b;

}

.header{

  padding: 28px 32px 20px;

  border-bottom: 1px solid #e2e8f0;

  background: #ffffff;

}

.title{

  font-size: 1.5rem;

  font-weight: 800;
```

```
margin: 0 0 4px 0;
color: #0f172a;
}

.subtitle {
font-size: 0.9rem;
color: #64748b;
margin: 0;
}

.layout {
display: grid;
grid-template-columns: 1fr 380px; /* Main content area and a 380px sidebar */
gap: 24px;
padding: 28px 32px;
max-width: 1200px; /* Constrains the layout width on large screens */
margin: 0 auto; /* Centers the layout horizontally */
}

@media (max-width: 768px) {
.layout {
grid-template-columns: 1fr; /* Stacks columns vertically on mobile */
padding: 16px;
}
}
```

CategorySummary.module.css

Found in folder: **/src/components**

This CSS code defines styles for a container and its components, including headings, lists, rows, and visual indicators like dots and warning badges. It uses flexbox for layout control and sets specific colors, spacing, and typography for each element. The styles are

designed to create a clean, structured list with items that can include colored dots, labels, totals, and warning icons.

```
.container {  
  margin-top: 20px; /* Adds spacing above the container */  
}  
  
.heading {  
  font-size: 0.9rem;  
  font-weight: 700;  
  color: #64748b;  
  text-transform: uppercase;  
  letter-spacing: 0.05em;  
  margin: 0 0 12px 0; /* Sets bottom margin for spacing below heading */  
}  
  
.list {  
  list-style: none; /* Removes default list bullets */  
  padding: 0;  
  margin: 0;  
  display: flex;  
  flex-direction: column; /* Stacks list items vertically */  
  gap: 8px; /* Adds consistent spacing between list items */  
}  
  
.row {  
  display: flex;  
  align-items: center; /* Vertically centers items within each row */  
  gap: 8px; /* Spacing between row elements */  
}
```

```
.dot{  
  flex-shrink: 0; /* Prevents dot from shrinking in flex layout */  
  width: 8px;  
  height: 8px;  
  border-radius: 50%; /* Makes the element a perfect circle */  
}  
  
.label {  
  flex: 1; /* Allows label to expand and fill available space */  
  font-size: 0.88rem;  
  color: #475569;  
}  
  
.total {  
  font-size: 0.88rem;  
  font-weight: 600;  
  color: #dc2626; /* Red color for total values */  
}  
  
.warning {  
  display: inline-flex;  
  align-items: center;  
  justify-content: center;  
  flex-shrink: 0;  
  width: 18px;  
  height: 18px;  
  border-radius: 50%; /* Circular warning badge */  
  background: #dc2626; /* Red background */  
  color: #ffffff;
```

```
font-size: 0.75rem;

font-weight: 700;

line-height: 1; /* Ensures text fits neatly in circle */

}
```

Dashboard.module.css

Found in folder: **/src/components**

This CSS styles a container for displaying financial statistics. It defines a clean, card-like layout with a heading and a grid of stat cards. The stat cards show income and expense values with distinct green and red colors.

```
.container {

  background: #ffffff;

  border-radius: 12px;

  padding: 24px;

  box-shadow: 0 1px 3px rgba(0, 0, 0, 0.08);

  height: 100%;

}

.heading{

  font-size: 1.1rem;

  font-weight: 700;

  color: #0f172a;

  margin: 0 0 16px 0;

  padding-bottom: 12px;

  border-bottom: 2px solid #f1f5f9;

}

.stats {

  display: grid;

  grid-template-columns: 1fr 1fr; /* Two-column grid for stat cards */
}
```

```
gap: 12px;
margin-bottom: 4px;
}
.statCard {
background: #f8fafc;
border-radius: 8px;
padding: 14px;
}
.statLabel {
font-size: 0.78rem;
font-weight: 600;
text-transform: uppercase;
letter-spacing: 0.05em;
color: #94a3b8;
margin: 0 0 6px 0;
}
.statValue {
font-size: 1.1rem;
font-weight: 700;
margin: 0;
}
.income {
color: #16a34a; /* Green for positive/income values */
}
.expense {
color: #dc2626; /* Red for negative/expense values */
```

```
}
```

MonthlyBalance.module.css

Found in folder: **/src/components**

This CSS code defines styles for financial display components, including containers for profit and loss values. It sets up visual distinctions with different background colors, borders, and text colors for profit (green theme) versus loss (red theme) states. The styles also format labels, amounts, and status text with specific typography and spacing rules.

```
.container {  
  border-radius: 10px;  
  padding: 18px;  
  text-align: center;  
  margin-bottom: 20px;  
}  
  
/* Profit theme - green color scheme */  
.profit {  
  background: #f0fdf4;  
  border: 1px solid #bbf7d0;  
}  
  
/* Loss theme - red color scheme */  
.loss {  
  background: #fef2f2;  
  border: 1px solid #fecaca;  
}  
  
/* Label styling for metric names */  
.label {  
  font-size: 0.82rem;  
  font-weight: 600;
```

```
text-transform: uppercase;

letter-spacing: 0.05em;

margin: 0 0 6px 0;

color: #64748b;
}

/* Main amount display */

.amount {

font-size: 1.6rem;

font-weight: 800;

margin: 0 0 4px 0;

color: inherit;
}

/* Profit-specific amount color */

.profit .amount {

color: #16a34a;
}

/* Loss-specific amount color */

.loss .amount {

color: #dc2626;
}

/* Status text styling */

.status {

font-size: 0.82rem;

font-weight: 500;

margin: 0;
}
```

```
/* Profit-specific status color */
```

```
.profit .status {  
  color: #16a34a;  
}
```

```
/* Loss-specific status color */
```

```
.loss .status {  
  color: #dc2626;  
}
```

TransactionItem.module.css

Found in folder: **/src/components**

This CSS code styles list items to display financial transactions with colored dots, descriptions, categories, and amounts. It arranges these elements horizontally with spacing and borders between items, except the last one. Income and expense amounts are colored green and red respectively.

```
.item {  
  display: flex;  
  align-items: center;  
  gap: 12px;  
  padding: 12px 0;  
  border-bottom: 1px solid #f1f5f9; /* Separator between items */  
  list-style: none;  
}  
.item:last-child {  
  border-bottom: none; /* Remove separator from last item */  
}  
.dot {  
  flex-shrink: 0; /* Prevent dot from shrinking */
```

```
width: 10px;

height: 10px;

border-radius: 50%; /* Circular dot */
}

.details {

flex: 1; /* Take up remaining space */

display: flex;

flex-direction: column;

gap: 2px;
}

.description {

font-size: 0.95rem;

font-weight: 500;

color: #1e293b;
}

.category {

font-size: 0.78rem;

color: #94a3b8; /* Muted category text */
}

.amount {

font-size: 0.95rem;

font-weight: 600;

white-space: nowrap; /* Prevent amount from wrapping */
}

.income {

color: #16a34a; /* Green for income */
```

```
}  
  
.expense {  
  color: #dc2626; /* Red for expense */  
}
```

TransactionList.module.css

Found in folder: **/src/components**

This CSS code defines styles for a container, heading, search label, search input, and list components. It sets visual properties like colors, borders, spacing, and interactive states for a search interface. The styles create a clean, card-like layout with focused input feedback.

```
.container {  
  background: #ffffff;  
  border-radius: 12px;  
  padding: 24px;  
  box-shadow: 0 1px 3px rgba(0, 0, 0, 0.08);  
  height: 100%;  
}  
  
.heading {  
  font-size: 1.1rem;  
  font-weight: 700;  
  color: #0f172a;  
  margin: 0 0 14px 0;  
  padding-bottom: 12px;  
  border-bottom: 2px solid #f1f5f9; /* Subtle separator below heading */  
}  
  
.searchLabel {  
  display: block;
```

```
margin-bottom: 6px;

font-size: 0.82rem;

font-weight: 600;

color: #475569;
}

.searchInput {

width: 100%;

margin-bottom: 16px;

border: 1px solid #cbd5e1;

border-radius: 8px;

padding: 10px 12px;

font: inherit;

color: #0f172a;

background: #ffffff;
}

.searchInput::placeholder {

color: #94a3b8; /* Placeholder text color */
}

.searchInput:focus {

border-color: #64748b;

outline: 3px solid #e2e8f0; /* Custom focus outline */
}

.list {

padding: 0;

margin: 0;
}
```

monthlyBudgets.js

Found in folder: **/src/data**

This code defines an object containing budget categories and their monthly limits, then exports it as the default export for use in other modules.

```
const monthlyBudgets = {  
  Housing: 900,    // Rent or mortgage  
  Food: 200,      // Groceries and dining  
  Entertainment: 50, // Movies, subscriptions  
  Utilities: 100,  // Electricity, water, internet  
  Health: 60,     // Medications, insurance  
  Shopping: 75,   // Clothing, miscellaneous  
  Transport: 75,  // Fuel, public transit  
};  
  
export default monthlyBudgets;
```

transactions.js

Found in folder: **/src/data**

This code defines an array of transaction objects with properties like description, amount, category, and month, then exports it as the default export. Each transaction represents either income (positive amount) or expense (negative amount) for the month of January. The exported array can be imported and used elsewhere in the project.

// Array of transaction objects for January

```
const transactions = [  
  { id: 1, description: 'Salary', amount: 3200, category: 'Income', month: 'January' },  
  { id: 2, description: 'Rent', amount: -950, category: 'Housing', month: 'January' },  
  { id: 3, description: 'Groceries', amount: -120, category: 'Food', month: 'January' },  
  { id: 4, description: 'Netflix', amount: -15, category: 'Entertainment', month: 'January' },  
  { id: 5, description: 'Electricity bill', amount: -85, category: 'Utilities', month: 'January' },  
];
```

```

    { id: 6, description: 'Freelance project', amount: 800, category: 'Income', month: 'January'
    },
    { id: 7, description: 'Restaurant dinner', amount: -65, category: 'Food', month: 'January' },
    { id: 8, description: 'Gym membership', amount: -40, category: 'Health', month: 'January' },
    { id: 9, description: 'Internet bill', amount: -35, category: 'Utilities', month: 'January' },
    { id: 10, description: 'Coffee shops', amount: -45, category: 'Food', month: 'January' },
    { id: 11, description: 'Amazon purchase', amount: -78, category: 'Shopping', month:
    'January' },
    { id: 12, description: 'Bus pass', amount: -60, category: 'Transport', month: 'January' },
  ];

// Export as default for use in other modules

export default transactions;

```

transactionService.js

Found in folder: **/src/services**

This code provides functions to analyze financial transaction data. It calculates totals, summaries, and alerts based on transaction categories and monthly budgets. The functions help track income, expenses, balances, and budget compliance.

```

import transactions from '../data/transactions.js';

import monthlyBudgets from '../data/monthlyBudgets.js';

export function getAllTransactions() {

  return transactions;

}

export function getTransactionsByCategory(category) {

  return transactions.filter((t) => t.category === category);

}

export function getTotalIncome() {

  return transactions

```

```

    .filter((t) => t.amount > 0) // Positive amounts represent income
    .reduce((sum, t) => sum + t.amount, 0);
}

export function getTotalExpenses() {
    return transactions
        .filter((t) => t.amount < 0) // Negative amounts represent expenses
        .reduce((sum, t) => sum + t.amount, 0);
}

export function getBalance() {
    return getTotalIncome() + getTotalExpenses(); // Net balance calculation
}

export function getSummaryByCategory() {
    const map = {};
    for (const t of transactions) {
        if (!map[t.category]) {
            map[t.category] = { category: t.category, total: 0 };
        }
        map[t.category].total += t.amount;
    }
    return Object.values(map);
}

export function getExpenseSummaryByCategory() {
    return getSummaryByCategory()
        .filter((item) => item.total < 0) // Only include expense categories
        .map((item) => {
            const spent = Math.abs(item.total);

```

```

const budgetLimit = monthlyBudgets[item.category] ?? null;

const isOverBudget = budgetLimit !== null && spent > budgetLimit;

return {
  ...item,
  spent,
  budgetLimit,
  isOverBudget,
  overBudgetBy: isOverBudget ? spent - budgetLimit : 0,
};
});
}

export function getBudgetAlertsByCategory() {
  return getExpenseSummaryByCategory().filter((item) => item.isOverBudget);
}

export function getTopSpendingCategory() {
  const [topCategory] = getExpenseSummaryByCategory().sort((a, b) => b.spent - a.spent);
  return topCategory.category;
}

export function getLowestSpendingCategory() {
  const [lowestCategory] = getExpenseSummaryByCategory().sort((a, b) => a.spent - b.spent);
  return lowestCategory.category;
}

export function getAverageDailySpend() {
  return Number((Math.abs(getTotalExpenses()) / 31).toFixed(2)); // Assumes 31-day month
}

```

transactionService.test.js

Found in folder: **/src/services**

This code tests various financial transaction service functions, verifying they return correct data like totals, category summaries, and budget alerts. It checks that functions handle edge cases, such as unknown categories, and validate specific expected values like the top spending category and average daily spend.

```
import { describe, it, expect } from 'vitest';

import {
  getAllTransactions,
  getTotalIncome,
  getTotalExpenses,
  getBalance,
  getTransactionsByCategory,
  getSummaryByCategory,
  getExpenseSummaryByCategory,
  getBudgetAlertsByCategory,
  getTopSpendingCategory,
  getAverageDailySpend,
} from './transactionService.js';

describe('transactionService', () => {

  it('getAllTransactions returns all 12 transactions', () => {
    expect(getAllTransactions()).toHaveLength(12);
  });

  it('getTotalIncome returns the correct sum', () => {
    expect(getTotalIncome()).toBe(4000);
  });

  it('getTotalExpenses returns the correct sum', () => {
```

```

    expect(getTotalExpenses()).toBe(-1493);
  });
  it('getBalance returns the correct balance', () => {
    expect(getBalance()).toBe(2507);
  });
  it('getTransactionsByCategory returns only matching transactions', () => {
    const food = getTransactionsByCategory('Food');
    expect(food).toHaveLength(3);
    expect(food.every((t) => t.category === 'Food')).toBe(true);
  });
  it('getTransactionsByCategory returns empty array for unknown category', () => {
    expect(getTransactionsByCategory('Unknown')).toEqual([]);
  });
  it('getSummaryByCategory returns correct number of categories', () => {
    expect(getSummaryByCategory()).toHaveLength(8);
  });
  it('getSummaryByCategory returns correct total for Food category', () => {
    const summary = getSummaryByCategory();
    const food = summary.find((item) => item.category === 'Food');
    expect(food.total).toBe(-230);
  });
  it('getExpenseSummaryByCategory returns only expense categories with budget status', ()
=> {
    const summary = getExpenseSummaryByCategory();
    expect(summary).toHaveLength(7);
    // Ensure all items are expenses (negative totals)

```

```
expect(summary.every((item) => item.total < 0)).toBe(true);

// Check that budget status is included

expect(summary.every((item) => typeof item.isOverBudget === 'boolean')).toBe(true);
});

it('getBudgetAlertsByCategory returns categories over budget', () => {
  expect(getBudgetAlertsByCategory()).toEqual([
    {
      category: 'Housing',
      total: -950,
      spent: 950,
      budgetLimit: 900,
      isOverBudget: true,
      overBudgetBy: 50,
    },
    {
      category: 'Food',
      total: -230,
      spent: 230,
      budgetLimit: 200,
      isOverBudget: true,
      overBudgetBy: 30,
    },
    {
      category: 'Utilities',
      total: -120,
      spent: 120,
```

```

    budgetLimit: 100,
    isOverBudget: true,
    overBudgetBy: 20,
  },
  {
    category: 'Shopping',
    total: -78,
    spent: 78,
    budgetLimit: 75,
    isOverBudget: true,
    overBudgetBy: 3,
  },
];
});

it('getTopSpendingCategory returns the category with the highest total expenses', () => {
  expect(getTopSpendingCategory()).toBe('Housing');
});

it('getAverageDailySpend returns the average expense spend over 31 days', () => {
  expect(getAverageDailySpend()).toBe(48.16);
});
});

```

categoryUtils.js

Found in folder: **/src/utils**

This code defines a mapping of category names to specific hex colour codes and provides a function to retrieve the colour for a given category, defaulting to a fallback colour if the category is not found.

```
const CATEGORY_COLOURS = {
```

```
Income: '#22c55e',
Housing: '#3b82f6',
Food: '#f97316',
Entertainment: '#a855f7',
Utilities: '#eab308',
Health: '#ef4444',
Shopping: '#06b6d4',
Transport: '#14b8a6',
};

const FALLBACK_COLOUR = '#6b7280'; // Default colour for unknown categories

export function getCategoryColour(category) {
  // Return the mapped colour or fallback if category not found
  return CATEGORY_COLOURS[category] ?? FALLBACK_COLOUR;
}
```

categoryUtils.test.js

Found in folder: **/src/utls**

This code tests a function that returns color strings for categories. It verifies that known categories receive a string color and that unknown categories get a specific fallback color.

```
import { describe, it, expect } from 'vitest';

import { getCategoryColour } from './categoryUtils.js';

// Predefined list of categories expected to have valid colors

const KNOWN_CATEGORIES = [
  'Income',
  'Housing',
  'Food',
  'Entertainment',
```

```

    'Utilities',
    'Health',
    'Shopping',
    'Transport',
  ];
  describe('categoryUtils', () => {
    // Ensures the function returns a string for each category in the known list
    it('getCategoryColour returns a string for every known category', () => {
      for (const category of KNOWN_CATEGORIES) {
        expect(typeof getCategoryColour(category)).toBe('string');
      }
    });
    // Tests that an unrecognized category receives the designated fallback color
    it('getCategoryColour returns a fallback value for an unknown category', () => {
      expect(getCategoryColour('Unknown')).toBe('#6b7280');
    });
  });
});

```

currencyUtils.js

Found in folder: **/src/utils**

This function formats a numeric amount as a British Pound currency string. It ensures the amount is displayed with two decimal places and includes thousand separators. A negative sign is prepended for negative amounts, while the pound symbol is always placed at the front.

```

export function formatCurrency(amount) {
  // Get absolute value to format digits independently of sign
  const abs = Math.abs(amount);

  // Format to 2 decimal places and insert thousand separators

```

```

const formatted = abs.toFixed(2).replace(/B(?=(\d{3})+(?!\d))/g, ',');

// Add currency symbol and restore negative sign if needed

return amount < 0 ? ` -£${formatted}` : ` £${formatted}`;
}

```

currencyUtils.test.js

Found in folder: **/src/utills**

This code tests the `formatCurrency` function to ensure it correctly formats positive and negative numbers into UK currency strings with pound symbols, commas for thousands, and two decimal places.

```

import { describe, it, expect } from 'vitest';

import { formatCurrency } from './currencyUtils.js';

// Test suite for currency formatting utilities

describe('currencyUtils', () => {

  // Tests formatting of positive numbers

  it('formatCurrency formats a positive number correctly', () => {

    expect(formatCurrency(3200)).toBe('£3,200.00');

  });

  // Tests formatting of negative numbers

  it('formatCurrency formats a negative number correctly', () => {

    expect(formatCurrency(-950)).toBe('-£950.00');

  });

});

```

Weather Dashboard

AGENTS.md

Found in folder: **/**

This file provides project documentation for a weather dashboard, outlining its purpose, technology stack, development commands, and coding conventions. It specifies the

project's structure and critical rules that must be followed during development. The document serves as a reference for students working on the codebase.

Weather Dashboard — AGENTS.md

Project

A React weather dashboard built as a starter project **for** a course on AI-assisted development. Students use **this** codebase to practice finding and fixing bugs with AI tooling.

Stack

- React 19 + Vite 7
- CSS Modules — no external UI libraries
- Vitest **for** unit testing

Commands

Command	Description
----- -----	
`npm run dev`	Start development server
`npm run build`	Production build
`npm run test`	Run Vitest test suite

Conventions

- All components live **in** `src/components/` with a co-located CSS module
- Shared utilities live **in** `src/utils/`
- Mock weather data lives **in** `src/data/`
- CSS **class** names use camelCase inside module files
- Component files use PascalCase; utility and data files use camelCase

Critical Rules

- **Do** not add external dependencies beyond React, Vite, and Vitest
- **Do** not add Tailwind or any CSS framework or component library
- **Do** not add routing or localStorage

- **Do** not replace the mock data with real API calls
- The broken behaviours **in this** repo are intentional student exercises — **do** not silently fix them
- Before the final commit **of** any session, run `/review` and **then** `git diff` to summarise every change made **in** plain English.

README.md

Found in folder: /

This is a README file that provides an overview of the Weather Dashboard project. It describes the application's purpose, lists supported cities, and gives setup instructions. It also outlines the project's technology stack and testing commands.

Weather Dashboard

A simple weather dashboard built with React and Vite. Type a city name into the search bar to view current weather conditions including temperature **in** both Celsius and Fahrenheit, wind speed, and sky condition.

Supported Cities

London, **New** York, Tokyo, Paris, Sydney

Searches are case-insensitive. Entering an unrecognised city shows a helpful error message.

Getting Started

```
```bash
```

```
npm install
```

```
npm run dev
```

```
```
```

Open [<http://localhost:5173>](http://localhost:5173) in your browser.

Running Tests

```
```bash
```

```
npm run test
```

```
```
```

Tech Stack

- [React](https://react.dev/) — UI library
- [Vite](https://vitejs.dev/) — build tool and dev server
- [Vitest](https://vitest.dev/) — unit testing framework
- Plain CSS Modules — no external UI libraries

SKILL.md

Found in folder: **/.codex/skills/debug-and-verify**

This code defines a structured workflow for debugging failing tests in a local project, guiding the user from analyzing test output to making incremental fixes and committing verified changes. It provides step-by-step instructions to identify failures, inspect code, apply focused fixes, and verify results through repeated test runs. The workflow concludes with steps to review changes and commit fixes only after all tests pass.

name: debug-and-verify

description: Use when given raw failing test output **for** a local project and asked to debug it. Fix one failure at a time, run npm run test after each fix, review the diff, and commit only after confirming all tests pass.

Debug And Verify

Workflow

Use **this** skill when the user provides raw `npm run test` output or asks to fix failing tests **in this** project.

1. Read the failure output carefully.

- Identify the failing file, test name, assertion or parser error, expected value, received value, and stack location.

- Start with the first concrete failure. **Do** not guess at unrelated bugs.

2. Inspect the smallest relevant code surface.

- Read the failing test and the implementation it exercises.

- Use ``rg`` first **for** related symbols, messages, and imports.
- Respect project instructions and avoid silently fixing unrelated intentional exercises.

3. Make one focused fix.

- Keep edits limited to the files needed **for** the current failure.
- Preserve existing style and project conventions.
- **Do** not add dependencies, routing, localStorage, real APIs, UI libraries, or CSS frameworks.

4. Verify incrementally.

- Run ``npm run test`` immediately after each fix.
- **If** tests still fail, treat the **new** output as the next input and repeat from step 1.
- Continue until ``npm run test`` exits successfully.

5. Review the diff before committing.

- Run a diff command such as ``git diff -- <changed-files>`` when the project is a git repo.
- Confirm the diff only contains the intended fixes.
- **If** unrelated user changes are present, leave them untouched and **do** not include them **in** the commit.

6. Commit the verified fix.

- Check git status.
- Stage only the files changed **for this** debugging task.
- Commit with a message that confirms tests pass, **for** example:

```
```bash
```

```
git commit -m "Fix failing tests; npm run test passes"
```

```
```
```

Reporting

In the final response, include:

- The files changed.
- The final ``npm run test`` result.

- The commit hash, **if** a commit was created.
- Any reason a commit could not be created, such as the project not being a git repo.

App.jsx

Found in folder: **/src**

This code provides a user interface for searching and displaying weather data. It manages loading states and errors while fetching weather information for a given city. The search functionality includes a debouncing mechanism to prevent outdated results from being displayed.

```
import { useRef, useState } from 'react'

import SearchBar from './components/SearchBar'

import WeatherCard from './components/WeatherCard'

import ErrorMessage from './components/ErrorMessage'

import LoadingSpinner from './components/LoadingSpinner'

import { getWeatherByCity } from './data/weatherData'

import styles from './App.module.css'

function App() {

  const [weatherData, setWeatherData] = useState(null)

  const [error, setError] = useState(false)

  const [isLoading, setIsLoading] = useState(false)

  // Tracks the most recent search to ignore outdated responses

  const latestSearchId = useRef(0)

  const handleSearch = (city) => {

    // Increment search ID to identify the current request

    const searchId = latestSearchId.current + 1

    latestSearchId.current = searchId

    setIsLoading(true)

    setWeatherData(null)
```

```

setError(false)

// Simulate network delay and implement debouncing
setTimeout(() => {

  // Ignore this response if a newer search has started
  if (searchId !== latestSearchId.current) {

    return

  }

  const data = getWeatherByCity(city)

  if (data) {

    setWeatherData(data)

    setError(false)

  } else {

    setWeatherData(null)

    setError(true)

  }

  setIsLoading(false)

}, 600)
}

return (

  <div className={styles.container}>

    <h1 className={styles.title}>Weather Dashboard</h1>

    <SearchBar onSearch={handleSearch} />

    {isLoading && <LoadingSpinner />}

    {!isLoading && error && <ErrorMessage />}

    {!isLoading && weatherData && <WeatherCard data={weatherData} />}

  </div>

```

```
)  
}
```

export default App

main.jsx

Found in folder: **/src**

This code initializes a React application by rendering the main App component into the DOM. It uses React's StrictMode to highlight potential problems during development. The App component is mounted inside the HTML element with the id 'root'.

```
import { StrictMode } from 'react';  
import { createRoot } from 'react-dom/client';  
import App from './App.jsx';  
  
// Create a React root for the DOM element with id 'root'  
const root = createRoot(document.getElementById('root'));  
  
// Render the App component inside StrictMode for development checks  
root.render(  
  <StrictMode>  
    <App />  
  </StrictMode>  
);
```

ErrorMessage.jsx

Found in folder: **/src/components**

This component renders a styled error message when a city search fails, suggesting example cities to try. It uses CSS Modules for styling by applying a class from the imported stylesheet. The component is then exported for use elsewhere in the application.

```
import styles from './ErrorMessage.module.css'  
  
function ErrorMessage() {  
  return (  
    <div>  
      <p>Error: City not found. Please try a different city.</p>  
      <p>Example cities: New York, Los Angeles, London, Paris, Tokyo.</p>  
    </div>  
  );  
}
```

```

// Displays a user-friendly error message with suggested city names

<div className={styles.error}>

  City not found. Try London, New York, Tokyo, Paris or Sydney.

</div>

)
}

```

export default ErrorMessage

LoadingSpinner.jsx

Found in folder: **/src/components**

This code defines a React component that displays a loading spinner with an accessible announcement while weather data is being fetched. It uses CSS modules for styling and includes ARIA attributes to properly communicate its state to screen readers. The component exports a single functional element that visually and accessibly indicates a loading process.

```

import styles from './LoadingSpinner.module.css'

function LoadingSpinner() {

  return (

    // Container with accessibility attributes to announce loading state

    <div className={styles.loading} role="status" aria-live="polite">

      {/* Visual spinner element hidden from screen readers */}

      <span className={styles.spinner} aria-hidden="true" />

      <span>Loading weather data...</span>

    </div>

  )

}

export default LoadingSpinner

```

SearchBar.jsx

Found in folder: **/src/components**

This code defines a React component that renders a search form with an input field and a submit button. It uses local state to track the input value and triggers a provided `onSearch` function when the form is submitted with a non-empty, trimmed value. The component prevents the default form submission behavior and clears any extra whitespace from the input before passing it to the search handler.

```
import { useState } from 'react'

import styles from './SearchBar.module.css'

// SearchBar component that takes an onSearch callback prop
function SearchBar({ onSearch }) {
  // State to manage the value of the search input field
  const [inputValue, setInputValue] = useState("")

  // Handles form submission: prevents default, validates input, and calls onSearch
  const handleSubmit = (e) => {
    e.preventDefault()

    // Only search if the trimmed input is not empty
    if (inputValue.trim()) {
      onSearch(inputValue.trim())
    }
  }

  return (
    // Form that triggers handleSubmit on submission
    <form className={styles.form} onSubmit={handleSubmit}>
      <input
        className={styles.input}
        type="text"
        placeholder="Enter city name..."
      />
    </form>
  )
}
```

```

      value={inputValue}
      onChange={(e) => setInputValue(e.target.value)}
    />

    <button className={styles.button} type="submit">
      Search
    </button>

  </form>
)
}

export default SearchBar

```

WeatherCard.jsx

Found in folder: **/src/components**

This code renders a weather card component displaying city, condition, temperature in Celsius and Fahrenheit, and wind speed. It converts the temperature from Celsius to Fahrenheit using a utility function. The component receives weather data as a prop and applies CSS module styles for presentation.

```

import { celsiusToFahrenheit } from '../utils/temperatureUtils'

import styles from './WeatherCard.module.css'

function WeatherCard({ data }) {

  // Convert temperature to Fahrenheit and round to nearest integer
  const fahrenheit = Math.round(celsiusToFahrenheit(data.temperature))

  return (

    <div className={styles.card}>

      <h2 className={styles.city}>{data.city}</h2>

      <p className={styles.condition}>{data.condition}</p>

      <div className={styles.temperature}>

        <span className={styles.tempC}>{data.temperature}°C</span>

```

```

    <span className={styles.tempF}>{fahrenheit}°F</span>
  </div>

  <div className={styles.wind}>
    <span className={styles.windLabel}>Wind Speed</span>
    <span className={styles.windValue}>{data.windSpeed} km/h</span>
  </div>
</div>
)
}

export default WeatherCard

```

index.html

Found in folder: /

This HTML file sets up the basic structure of a web page, including metadata and viewport settings for responsive design. It links a favicon and loads a React application by mounting it into a `div` element with the id “root”. The page’s JavaScript entry point is specified as a module script.

```

<!doctype html>

<html lang="fr">

  <head>

    <meta charset="UTF-8" />

    <link rel="icon" type="image/svg+xml" href="/vite.svg" />

    <meta name="viewport" content="width=device-width, initial-scale=1.0" />

    <title>Vite + React</title>

  </head>

  <body>

    <!-- Root container where the React app will be rendered -->

    <div id="root"></div>

```

```
<!-- Entry point for the React application built with Vite -->
<script type="module" src="/src/main.jsx"></script>
</body>
</html>
```

App.module.css

Found in folder: **/src**

This CSS defines two classes: `.container` creates a full-height, centered column layout with padding and a light background, while `.title` styles a large, bold, dark heading with bottom margin.

```
.container {
  min-height: 100vh; /* Ensures container fills at least the full viewport height */
  display: flex;
  flex-direction: column;
  align-items: center;
  padding: 40px 20px;
  background-color: #f0f4f8;
  font-family: 'Segoe UI', system-ui, sans-serif;
  box-sizing: border-box; /* Padding included in width/height calculation */
}

.title {
  font-size: 2rem;
  color: #1a202c;
  margin: 0 0 32px; /* Top, horizontal, bottom margin */
  font-weight: 700;
}
```

ErrorMessage.module.css

Found in folder: **/src/components**

This CSS defines a visual style for error messages, making them stand out with a red color scheme and rounded borders. It ensures the error container is responsive but doesn't exceed a maximum width. The styling includes padding, a light red background, and a darker red border for clear visual feedback.

```
.error {  
  width: 100%; /* Takes full width of its container */  
  max-width: 480px; /* But doesn't exceed this maximum width */  
  padding: 16px;  
  background-color: #fff5f5; /* Light red background */  
  border: 1px solid #feb2b2; /* Medium red border */  
  border-radius: 8px; /* Rounded corners */  
  color: #c53030; /* Dark red text color */  
  font-size: 0.9rem;  
  text-align: center;  
}
```

LoadingSpinner.module.css

Found in folder: **/src/components**

This CSS defines a loading container with a centered spinner animation. The `.loading`` class creates a styled box with flex alignment and shadow, while `.spinner`` creates an animated circular element using a rotating border. The `@keyframes spin`` rule continuously rotates the spinner to indicate loading activity.

```
.loading {  
  width: 100%;  
  max-width: 400px;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  gap: 12px; /* Space between spinner and text */
```

```
padding: 24px;

background: white;

border-radius: 16px;

box-shadow: 0 4px 24px rgba(0, 0, 0, 0.1);

color: #2d3748;

font-size: 0.95rem;

font-weight: 600;

box-sizing: border-box; /* Include padding in width calculation */
}

.spinner {

width: 24px;

height: 24px;

border: 3px solid #bee3f8; /* Light blue background ring */

border-top-color: #4299e1; /* Dark blue animated segment */

border-radius: 50%;

animation: spin 0.8s linear infinite; /* Continuous rotation */

flex: 0 0 auto; /* Prevent spinner from stretching */

}

@keyframes spin {

to {

transform: rotate(360deg); /* Full rotation */

}

}

}
```

SearchBar.module.css

Found in folder: **/src/components**

This CSS code styles a form container, an input field, and a button. It defines their layout, visual appearance, and interactive states like focus and hover. The styles create a responsive, modern UI with smooth transitions.

```
.form {  
  display: flex;  
  gap: 8px; /* Spacing between child elements */  
  width: 100%;  
  max-width: 480px; /* Constrains form width on larger screens */  
  margin-bottom: 24px; /* Space below the form */  
}  
  
.input {  
  flex: 1; /* Allows input to grow and fill available space */  
  padding: 12px 16px;  
  font-size: 1rem;  
  border: 2px solid #cbd5e0;  
  border-radius: 8px;  
  outline: none;  
  transition: border-color 0.2s; /* Smooth color change on focus */  
  font-family: inherit;  
}  
  
.input:focus {  
  border-color: #4299e1; /* Highlight color when input is focused */  
}  
  
.button {  
  padding: 12px 24px;  
  font-size: 1rem;  
  font-weight: 600;
```

```
color: white;

background-color: #4299e1;

border: none;

border-radius: 8px;

cursor: pointer;

transition: background-color 0.2s; /* Smooth color change on hover */

font-family: inherit;
}

.button:hover {

background-color: #3182ce; /* Darker shade on hover */

}
```

WeatherCard.module.css

Found in folder: **/src/components**

This CSS styles a weather display card with a clean, centered layout. It defines visual styles for the city name, condition, temperature in Celsius and Fahrenheit, and a wind information section. The card has a white background, rounded corners, and a subtle shadow.

```
.card {

width: 100%;

max-width: 400px;    /* Limits card width on larger screens */

background: white;

border-radius: 16px; /* Creates rounded corners */

padding: 32px;

box-shadow: 0 4px 24px rgba(0, 0, 0, 0.1); /* Adds a soft shadow */

text-align: center;

}

.city {
```

```
font-size: 1.75rem;

font-weight: 700;

color: #1a202c;

margin: 0 0 8px;    /* Bottom margin only */
}

.condition {

font-size: 1rem;

color: #718096;

margin: 0 0 24px;    /* Bottom margin only */
}

.temperature {

display: flex;

justify-content: center;

gap: 24px;          /* Space between Celsius and Fahrenheit */

margin-bottom: 24px;
}

.tempC {

font-size: 2.5rem;

font-weight: 700;

color: #2d3748;
}

.tempF {

font-size: 2.5rem;

font-weight: 300;    /* Lighter weight to differentiate from Celsius */

color: #a0aec0;
}
```

```
.wind {  
  display: flex;  
  justify-content: space-between; /* Separates label and value */  
  padding: 16px;  
  background: #f7fafc; /* Light background for the wind section */  
  border-radius: 8px;  
}  
  
.windLabel {  
  color: #718096;  
  font-size: 0.875rem;  
}  
  
.windValue {  
  color: #2d3748;  
  font-weight: 600;  
  font-size: 0.875rem;  
}
```

weatherData.js

Found in folder: **/src/data**

This code defines a set of hardcoded weather data for five cities and exports a function to retrieve that data. The `getWeatherByCity` function takes a city name as input, converts it to lowercase, and returns the corresponding weather object. If the city is not found in the data, it returns `null`.

```
const weatherData = {  
  london: {  
    city: 'London',  
    temperature: 18,  
    windSpeed: 12,
```

```
    condition: 'Partly Cloudy',
  },
  'new york': {
    city: 'New York',
    temperature: 24,
    windSpeed: 8,
    condition: 'Sunny',
  },
  tokyo: {
    city: 'Tokyo',
    temperature: 21,
    windSpeed: 15,
    condition: 'Overcast',
  },
  paris: {
    city: 'Paris',
    temperature: 16,
    windSpeed: 10,
    condition: 'Light Rain',
  },
  sydney: {
    city: 'Sydney',
    temperature: 28,
    windSpeed: 20,
    condition: 'Clear Sky',
  },
```

```

}

// Returns weather data for a given city name, or null if not found
export const getWeatherByCity = (cityName) => {

  // Lookup is case-insensitive; convert input to lowercase to match object keys

  return weatherData[cityName.toLowerCase()] ?? null
}

```

weatherData.test.js

Found in folder: **/src/data**

This code tests the `getWeatherByCity` function using the Vitest framework. It verifies that the function returns correct weather data for a known valid city. It also checks that the function returns null when given an invalid city name.

```

import { describe, expect, it } from 'vitest'

import { getWeatherByCity } from './weatherData'

// Test suite for the getWeatherByCity function
describe('getWeatherByCity', () => {

  // Tests the function with a valid city input
  it('returns weather data for a valid city', () => {
    expect(getWeatherByCity('London')).toEqual({
      city: 'London',
      temperature: 18,
      windSpeed: 12,
      condition: 'Partly Cloudy',
    })
  })

  // Tests the function with an invalid city input
  it('returns null for an invalid city', () => {
    expect(getWeatherByCity('Atlantis')).toBeNull()
  })
}

```

```
}}
```

```
}}
```

temperatureUtils.js

Found in folder: **/src/Utils**

This function converts a temperature from Celsius to Fahrenheit. It takes a Celsius value as input and returns the equivalent Fahrenheit value using the standard conversion formula.

```
export const celsiusToFahrenheit = (celsius) => {  
  
  // Convert Celsius to Fahrenheit using the formula:  $(C \times 9/5) + 32$   
  
  return celsius * 1.8 + 32  
  
}
```

temperatureUtils.test.js

Found in folder: **/src/Utils**

This code tests a temperature conversion function by verifying that 100 degrees Celsius correctly converts to 212 degrees Fahrenheit. It uses a testing framework to define a test case and an assertion to check the result.

```
import { describe, it, expect } from 'vitest'  
  
import { celsiusToFahrenheit } from './temperatureUtils'  
  
// Test suite for the celsiusToFahrenheit conversion function  
  
describe('celsiusToFahrenheit', () => {  
  
  // Tests a specific conversion: 100°C should equal 212°F  
  
  it('converts 100°C to 212°F', () => {  
  
    expect(celsiusToFahrenheit(100)).toBe(212)  
  
  })  
  
})
```